40. Инкремент, декремент и побочные эффекты

На этом уроке мы рассмотрим, что такое инкремент и декремент в языке C++, а также разберемся с таким понятием, как «побочные эффекты».

Оглавление:

1. **Инкремент и декремент**
2. **Побочные эффекты**

Инкремент и декремент

Операции **инкремента** (увеличение на `1`) и **декремента** (уменьшение на `1`) переменных настолько используемые, что у них есть свои собственные операторы в языке C++. Кроме того, каждый из этих операторов имеет две версии применения: префикс и постфикс.

Оператор	Символ	Пример	Операция
Префиксный инкремент (пре-инкремент)	++	++x	Инкремент x, затем вычисление x
Префиксный декремент (пре-декремент)	−−	−−x	Декремент x, затем вычисление x
Постфиксный инкремент (пост-инкремент)	++	x++	Вычисление x, затем инкремент x
Постфиксный декремент (пост-декремент)	−−	x−−	Вычисление x, затем декремент x

С операторами инкремента/декремента версии префикс всё просто. Значение переменной `x` сначала увеличивается/уменьшается, а затем уже вычисляется. Например:

```
int x = 5;
int y = ++x; // x = 6 и 6 присваивается переменной y
```

А вот с операторами инкремента/декремента версии постфикс несколько сложнее. Компилятор создает временную копию переменной `x`, увеличивает или уменьшает оригинальный `x` (не копию), а затем возвращает копию. Только после возврата копия `x` удаляется. Например:

```
int x = 5;
int y = x++; // x = 6, но переменной y присваивается 5
```

Рассмотрим вышеприведенный код детально. Во-первых, компилятор создает временную копию `x`, которая имеет то же значение, что и оригинал (`5`). Затем увеличивается первоначальный `x` с `5` до `6`. После этого компилятор возвращает временную копию, значением которой является `5`, и присваивает её переменной `y`. Только после этого копия `x` уничтожается. Следовательно, в вышеприведенном примере мы получим `y = 5` и `x = 6`.

Вот еще один пример, показывающий разницу между версиями префикс и постфикс:

```
1  #include <iostream>
2
3  int main()
4  {
5      int x = 5, y = 5;
6      std::cout << x << " " << y << std::endl;
7      std::cout << ++x << " " << --y << std::endl; // версия префикс
8      std::cout << x << " " << y << std::endl;
9      std::cout << x++ << " " << y-- << std::endl; // версия постфикс
10     std::cout << x << " " << y << std::endl;
11
12     return 0;
13 }
```

Результат выполнения программы:

```
5 5
6 4
6 4
6 4
7 3
```

В строке №7 переменные `x` и `y` увеличиваются/уменьшаются на единицу непосредственно перед обработкой компилятором, так что сразу выводятся их новые значения. А в строке №9 временные копии (`x = 6` и `y = 4`) отправляются в **cout**, а только после этого исходные `x` и `y` увеличиваются/уменьшаются на единицу. Именно поэтому изменения значений переменных после выполнения операторов версии постфикс не видно до следующей строки.

Версия префикс увеличивает/уменьшает значения переменных перед обработкой компилятором, версия постфикс — после обработки компилятором.

Правило: Используйте префиксный инкремент и префиксный декремент вместо постфиксного инкремента и постфиксного декремента. Версии префикс не только более производительны, но и ошибок с ними (по статистике) меньше.

Побочные эффекты

Функция или выражение имеет **побочный эффект**, если она/оно изменяет состояние чего-либо, делает ввод/вывод или вызывает другие функции, которые имеют побочные эффекты.

В большинстве случаев побочные эффекты являются полезными:

```
#include <iostream>

int main()
{
    int x = 5;
    ++x;
    std::cout << x;

    return 0;
}
```

В примере, приведенном выше, оператор присваивания имеет побочный эффект, который проявляется в изменении значения переменной `x`. Оператор `++` имеет побочный эффект инкремента переменной `x`. Вывод `x` имеет побочный эффект внесения изменений в консольное окно.

Также побочные эффекты могут приводить и к неожиданным результатам:

```
#include <iostream>

int add(int x, int y)
{
    return x + y;
}

int main()
{
    int x = 5;
    int value = add(x, ++x); // здесь 5 + 6 или 6 + 6?  Это зависит от компилятора и от того, в каком порядке он будет обрабат

    std::cout << value; // результатом может быть 11 или 12

    return 0;
}
```

Язык C++ не определяет порядок, в котором вычисляются аргументы функции. Если левый аргумент будет вычисляться первым, то `add(5, 6)` и результат — `11`. Если правый аргумент будет вычисляться первым, то `add(6, 6)` и результат — `12`! А проблема то кроется в побочном эффекте одного из аргументов функции add().

Вот еще один пример:

```
#include <iostream>

int main()
{
    int x = 1;
    x = x++;
    std::cout << x;

    return 0;
}
```

Какой результат выполнения этой программы? Если инкремент переменной `x` выполняется до операции присваивания, то ответ — `1`. Если же после операции присваивания, то ответ — `2`.

Есть и другие случаи, в которых C++ не определяет порядок обработки данных, поэтому в разных компиляторах могут быть разные результаты. Но даже в тех случаях, когда C++ и уточняет порядок обработки данных, некоторые компиляторы все равно вычисляют переменные с побочными эффектами некорректно. Этого всего можно избежать, если использовать переменные с побочными эффектами не более одного раза в одном стейтменте.

Правило: Не используйте переменную с побочным эффектом больше одного раза в одном стейтменте.